

# Pomfret Astro Technical Documentation

Pomfret Astro

Version 6.3.3

## Contents

|                                                          |          |
|----------------------------------------------------------|----------|
| <b>Pomfret Astro -- Technical Documentation</b>          | <b>1</b> |
| 1. System overview . . . . .                             | 2        |
| 2. Nightly imaging flow . . . . .                        | 2        |
| 3. Repository structure . . . . .                        | 3        |
| 4. Authentication and trust boundaries . . . . .         | 3        |
| 5. Persistence . . . . .                                 | 4        |
| 6. Observatory mode and status . . . . .                 | 4        |
| 7. Weather decisions . . . . .                           | 4        |
| 8. Scheduling and reconcile . . . . .                    | 5        |
| 9. Sessions, projects, and the board . . . . .           | 6        |
| 10. NINA delivery . . . . .                              | 6        |
| 11. Emergency stop and end night . . . . .               | 7        |
| 12. Projects, mosaics, and variable stars . . . . .      | 7        |
| 13. Moon avoidance . . . . .                             | 7        |
| 14. All-sky camera and ASC AI . . . . .                  | 8        |
| 15. Mount telemetry and Remote 3D panel . . . . .        | 8        |
| 16. Object storage and retention . . . . .               | 8        |
| 17. Live updates . . . . .                               | 8        |
| 18. Cron and the Windows agent . . . . .                 | 9        |
| 19. Production configuration . . . . .                   | 9        |
| 20. HTTP API surface . . . . .                           | 9        |
| 21. Development, testing, and deployment . . . . .       | 10       |
| 22. Delivery priority and weather gate summary . . . . . | 10       |

## Pomfret Astro -- Technical Documentation

**Version:** v6.3.3

**Production:** <https://www.pomfretastro.org>

**Repository:** <https://github.com/QimingTian/Pomfret-Astro-Website>

This document describes the architecture and operating rules of the Pomfret Astro observatory stack: the cloud application, the scheduling and safety logic, the Windows NINA agent, and the all-sky camera services that support autonomous imaging nights.

---

## 1. System overview

Pomfret Astro is an autonomous school observatory. Members submit imaging requests through a Next.js web application hosted on Vercel. A server-side scheduler decides what can run tonight under weather and altitude constraints. A Windows PC at the dome runs a Python agent that polls the cloud for NINA sequences, powers equipment through a Digital Loggers PDU, executes those sequences, and uploads results to Cloudflare R2. A Raspberry Pi beside the dome provides the all-sky camera stream and the ASC cloud and rain models used by weather decisions.

The observatory site is fixed in code at approximately 41.886 degN, 71.965 degW, in the America/New\_York time zone. For both scheduling and delivery, a target must remain at least **30 deg** above the horizon for the entire session.

Durable imaging state that must survive serverless cold starts is stored in Upstash Redis over REST. Session files and live previews are stored in R2. The website process itself retains almost no durable imaging state on the local filesystem.

---

## 2. Nightly imaging flow

A member creates a queue request. The row begins as **pending**. Periodically--from cron, weather refreshes, or session events--the server runs **reconcile**. Reconcile evaluates tonight's weather-permitted intervals, the altitude path of each target, moon avoidance, and FIFO ownership of the sky. When a request fits, it becomes **scheduled** and receives a **planned start** time (`plannedStartIso`).

The Windows agent polls `GET /api/imaging/nina-sequence` with the queue bearer secret. Work is delivered only when the observatory is **Ready**, the planned start is due, the target altitude is sufficient, and no higher-priority work such as **ESTOP** is pending. On delivery, a normal queue row moves to **in progress**. The agent writes the NINA JSON, warms the PDU, launches NINA, and reports progress through the session-progress webhook.

When the session finishes, the board and queue are marked **completed**. Reconcile may place the next work. If tonight's imaging is exhausted, the system may arm an **end-night** sequence that closes the dome. The agent post-processes frames as configured, uploads to R2, and reports object keys so members can download through short-lived presigned URLs.

If weather becomes unsafe, an administrator arms **ESTOP**, or a session fails in a way that requires locking the site, the same delivery channel carries an emergency-stop sequence instead of ordinary imaging work. That path is defined in Section 11.

### 3. Repository structure

The Next.js application under `app/` provides the member and admin dashboards (Weather, Plan, Remote, Gallery, Admin, Account) and the HTTP API under `app/api/`. Remote and Plan UI components live primarily in `components/remote/` and `components/plan/`.

Domain logic resides in `lib/`. Imaging is concentrated under `lib/imaging/`: the multi-night project planner, queue store and reconcile, session board, hold and release, emergency stop, weather-safety ESTOP, and NINA JSON helpers. Related modules outside that package include `lib/asc-cloud.ts` for the **Ready** gate, `lib/tonight-weather-gate.ts` for schedule weather, `lib/observatory-status-store.ts` for mode and status, `lib/schedule-strip.ts` and `lib/sunrise-window.ts` for night windows, and `lib/moon-avoidance.ts` for filter-dependent moon separation. Persistence helpers are in `lib/kv-rest.ts`. R2 download and object maps are in `lib/r2-session-download.ts`.

On site, `observatory/nina_agent.py` is the Windows poller. `observatory/camera_service.py` serves the all-sky MJPEG stream and status endpoints. `observatory/asc_cloud_ai.py` runs the Teachable Machine cloud and rain models. The repository-root templates `ESTop.json` and `End Night Session.json` are the NINA sequences the cloud patches and delivers for emergency close and normal end-of-night shutdown.

Legacy import paths such as `lib/imaging-emergency-stop.ts` re-export the current module layout for compatibility.

---

### 4. Authentication and trust boundaries

Members authenticate with an HTTP-only cookie named `pomfret_session`. Sessions and user records are stored in KV. A user is an imaging **admin** when `role` is `admin` or when the email appears in `BOOTSTRAP_ADMIN_EMAILS`. Admin-only imaging actions--**ESTOP**, observatory PATCH, session control, audit log, and schedule-control mutations--use that same admin check.

The observatory agent does not use a member cookie. It authenticates with `IMAGING_QUEUE_SECRET` as a Bearer token on sequence poll, agent pulse, session-files, and related agent routes. Cron uses `CRON_SECRET`. The NINA progress webhook may use Basic authentication with `NINA_SESSION_PROGRESS_BASIC`. Mount telemetry uses a dedicated secret. In production, missing required secrets **fail closed**; local development is more permissive.

Session-scoped routes such as download, edit, and delete allow the owning member, an admin, or an explicit session password header. Middleware does not perform login checks. It sets security headers (CSP, HSTS in production, framing and permissions policies) and excludes static and sky-data paths from its matcher.

## 5. Persistence

When `KV_REST_API_URL` and `KV_REST_API_TOKEN` are set, Upstash is the source of truth for queues, projects, the session board, **ESTOP**, observatory status, the audit log, R2 object maps, admin closed windows, and live-bus fan-out lists. Without KV, selected paths may fall back to files (for example `IMAGING_QUEUE_FILE`), but production imaging assumes KV.

Queue rows and projects are rewritten by reconcile and delivery. The **session board** is a parallel inventory of in-progress and terminal sessions for the Remote dashboard. **ESTOP** and **end-night** flags are stored separately so shutdown state is not conflated with ordinary scheduling. Keys under the `live:` prefix provide ephemeral fan-out for progress, preview, mount, site, and agent-wake signals.

---

## 6. Observatory mode and status

The observatory runs in either **manual** or **auto** mode. In **manual** mode, the operator-selected status is authoritative. In **auto** mode, the server recomputes status from daytime windows, weather, agent heartbeat, and whether NINA is currently reported running.

**Ready** means weather and connectivity allow work and the agent may receive sequences. **Busy** means a fresh NINA-running pulse arrived within about ninety seconds. **Disconnected** means the agent has not been heard from within that same window. **Closed -- Weather** means the **Ready** gate failed. **Closed -- Daytime** covers nautical dawn through nautical dusk. **Closed -- Maintenance** covers operator lockouts, admin closed windows, and the **ESTOP** lock.

Status computation syncs from KV first, then applies a hard **ESTOP** guard: while emergency stop is blocking, the site is forced to **manual** and **Closed -- Maintenance**, and a stale remote `auto` value cannot undo that lock. After that guard, admin closed windows, manual status, daytime, and the **Ready** weather evaluation apply in order. **Disconnected** and **Busy** may still override the computed base when the agent is missing or NINA is running.

Administrators may PATCH mode and status. Leaving the maintenance lock while **ESTOP** is in the **stopped** phase clears the ESTOP record, releases holds (including failed-sub-tonight auto-holds), and forces schedule reconcile.

---

## 7. Weather decisions

Weather participates in three independent decisions, plus a display-only astro layer.

The **Ready** gate decides whether the observatory may accept work at the present moment. When the ASC gate applies, ASC cloud must be below **20%** and rain must not be detected. When ASC is stale or an all-sky sequence is active, the gate falls back to Open-Meteo cloud cover below **10%**. Wind must remain under

**10 m/s**, and precipitation probability must remain at or below **20%**. Since v6.3.1, 7Timer transparency and seeing are never **Ready** gates; they appear only on the all-sky overlay and are highlighted when the scale is **5** or worse.

The **tonight schedule** gate decides which hours may receive planned sessions. It uses hourly Open-Meteo near the site. An hour is permitted only when cloud cover is under **10%**, precipitation probability is under **10%**, and wind is at most **10 m/s**. Across the imaging night a global hard block also applies: every counting hour must keep precip under **10%**, at most **three** hours may exceed **10 m/s** of wind, and at least **two consecutive** hours must pass all three checks. Hours that have already fully ended after night start no longer veto the remainder of the night. Each candidate session requires at least **80%** of its duration inside permitted intervals. The Remote tonight-permitted header further restricts its counting window to nautical dusk through nautical dawn.

**Weather-safety ESTOP** is a separate safety loop that arms only during nautical night, with a **45 s** debounce. It arms if Open-Meteo reports thunderstorm weather codes **95**, **96**, or **99** on a **20 km** ring (excluding the site center) for the current or next hour; if the site's current-hour precip probability exceeds **20%**; or if ASC reports rain detected with confidence at least **0.99** while the ASC gate applies. Arming places remaining work on hold, fails in-progress sessions, locks the observatory, and suppresses the activity-only end-night fallback.

When that stop was armed by weather safety and has reached the **stopped** phase, the same loop may unlock automatically if Open-Meteo is available, ASC is applicable, and none of the arm threats remain. Holds are released and mode returns to **auto**. Manual **ESTOP** and session-failure **ESTOP** never auto-clear. A sensor outage never counts as clear.

---

## 8. Scheduling and reconcile

The Remote "tonight" strip runs from **4:00 PM** local time to **8:00 AM** the following morning. The **night key** is the calendar day on which that strip starts. Scheduling windows and nautical daytime closure use nautical dusk and dawn (zenith angle **102 deg**). Open-Meteo nightly weather bounds follow sunset to the following sunrise for hourly samples.

**Reconcile** is the scheduling orchestrator. It debounces for **15 s** unless forced, performs no work while **ESTOP** is blocking, drops stale sub-sessions from prior nights, and unschedules pending work when weather is unknown or globally hard-blocked. Otherwise it first maintains any project already on the board, builds free time by subtracting altitude holds, admin force-runs, and existing occupancy, then walks pending rows in creation order. Project rows use the multi-night planner; ordinary queue rows use schedule insight. Reconcile still runs when the pending list is empty because in-progress projects may require replan. Completions, holds, weather routes, observatory changes, and cron may all trigger it.

The planner adds overhead--approximately **40 minutes** base, **5 minutes** per extra filter, and **10 minutes** for a meridian flip, with a shorter **20-minute** total on the variable-star queue path--then searches free windows

in **5-minute** steps. Altitude must remain at least **30 deg** for the entire session. Weather coverage must reach **80%**. If no full slot exists, frame counts may shrink to fit. Parent projects remain **pending** until NINA delivery marks them on board; only the head of the pending-project FIFO receives new tonight sub-sessions, and an active on-board project holds altitude-qualified sky. After a failed sub-session tonight, new subs for that project go **on hold** unless an admin force-run is active or the row was manually restored. Clearing **ESTOP** also releases those failed-sub-tonight holds.

Mosaic projects interleave panels with moon-aware placement, selecting the earliest workable start and preferring more frames on ties. Ordinary queue scheduling uses the same altitude, weather, step, and moon rules; variable-star rows skip moon blocking. Delivery is gated by planned start: the agent receives a session only when the current time is at or after `plannedStartIso`. Earlier starts occur only when reconcile advances the planned start, not merely because a brief **Ready** window appeared.

---

## 9. Sessions, projects, and the board

A queue row progresses through **pending**, **scheduled**, **on hold**, **in progress**, **completed**, **failed**, or **rejected**. A project parent remains **pending** until the first sub-session is delivered, then may become **in progress**, and becomes **completed** when no frames remain. Project night sub-sessions are **planned**, **scheduled**, **on hold**, **in progress**, **completed**, or **failed**. Holds originate from **ESTOP**, manual action, or the failed-sub-tonight rule. Releasing a project sub-session always returns it to **planned** so reconcile can retain the same sub index.

The **session board** is the Remote inventory of in-progress and terminal sessions. It is capacity-limited and also stores download timestamps, schedule-bar placement, and session password hashes used for shared access.

---

## 10. NINA delivery

The agent poll endpoint applies a fixed priority. An **ESTOP** that is **stopping** and not yet delivered is delivered first. While **ESTOP** remains blocking, or while NINA is freshly reported running, the server returns **409** so ordinary imaging cannot interrupt. Admin force-runs, closed windows, on-board project sub-sessions, and due **scheduled** queue rows follow in priority. If no imaging work remains, an armed **end-night** sequence may be delivered; otherwise the agent waits.

Progress posts from NINA append lines on the session, complete sessions when the end marker appears, and for **ESTOP** treat a dome-closed signal as stop completion and confirmation of the maintenance lock. Session completion evaluates whether **end-night** should become due after reconcile. Discord text inside NINA JSON trees is patched at delivery time so weather-safety and manual stops can present different messages while sharing the same template files.

## 11. Emergency stop and end night

**ESTOP** is a compare-and-swap state machine in KV with phases **stopping** and **stopped**, a queue id, the sessions held at arm time, and timestamps for delivery and dome-closed completion. An undelivered **stopping** state older than **six hours** is treated as stale and cleared.

There are three arm paths. An administrator may arm **ESTOP** from the dashboard. **Weather-safety ESTOP** may arm automatically under the storm, precip, or ASC rain rules in Section 7. A genuine session failure may lock the observatory and arm **ESTOP** so the dome still closes, except when the failure reason is already an emergency stop or an intentional delivery handoff. On arm, the site locks immediately to **manual** and **Closed -- Maintenance**, end-night due flags are cleared, activity-only end-night is suppressed for that night, and in-progress work is failed.

Clearing is either manual--an administrator leaves the maintenance lock--or, for weather-safety **ESTOP** only, automatic once the phase is **stopped** and both Open-Meteo and ASC report clear conditions. Holds release and scheduling resumes.

**End night** is separate from **ESTOP**. After the last real session of the night is consumed, or at nautical dawn, the agent may receive an end-night sequence that closes the dome and posts the ordinary completion Discord message. After an **ESTOP**, the activity-only fallback is suppressed so clearing the lock does not falsely announce that tonight's session completed.

---

## 12. Projects, mosaics, and variable stars

**Project mode** stores a parent record with remaining frames by filter and tonight's filter plan. Sub-session identifiers follow the form `{projectId}:night-{index}`. **Mosaic mode** adds panels and remaining frames per panel. The Plan page Framing tools build grids or custom panels with a NINA-aligned VIEW projection, and the planner interleaves panels across the night.

Variable stars use a VSX-derived catalog under `lib/variable-star/`, with Remote filters for types and periods and APIs for listing and lookup. Those sessions do not apply moon avoidance. A biweekly GitHub Action regenerates the shortlist via `npm run build:variable-star-shortlist`.

---

## 13. Moon avoidance

Moon avoidance uses a Lorentzian separation model compatible with ACP and NINA. At full moon, broadband LRGB requires approximately **110 deg** of separation, OIII approximately **95 deg**, SII approximately **65 deg**, and Ha approximately **55 deg**. The width is **fourteen days** (half a lunation). If the moon is below the horizon the check passes. If the moon is under **10 deg** altitude, the required separation is halved. Variable-star sessions skip the check entirely.

---

## 14. All-sky camera and ASC AI

The Pi serves the MJPEG stream at `https://cam.pomfretastro.org/camera/stream` and exposes camera and sequence status endpoints consumed by the cloud. ASC inference returns cloud cover percent, a rain detection with confidence and label, a day or night model phase, and staleness metadata. When a long all-sky sequence is running, the **Ready** and weather-safety paths treat ASC as not applicable so a frozen sky frame cannot falsely gate or arm the observatory. Auto exposure and white balance history may be recorded for administrators through the camera auto-tuning API.

---

## 15. Mount telemetry and Remote 3D panel

The observatory posts mount pointing samples to the cloud. Those samples fan out over a live stream and drive the Remote Telescope Status panel, which loads Three.js models from `public/telescope-models/`. Posting is authenticated with the mount telemetry secret.

---

## 16. Object storage and retention

Session outputs are stored under an R2 prefix (default `imaging`) keyed by queue id. Live previews use a fixed live-preview object per session. The agent uploads with boto3-compatible credentials, then posts `session-files` so KV records the object map. Members download through `GET /api/imaging/download`, which issues a short-lived presigned URL (default **five minutes**) and records that the board entry was downloaded.

Completed board rows and terminal projects are purged on the order of **48 hours**. Member session history is retained longer (approximately **60 days**). Resolved gallery submissions expire after approximately **30 days**. Cleanup runs through the cron-protected cleanup route and shared maintenance helpers.

---

## 17. Live updates

The Remote dashboard uses adaptive HTTP polling against `site-poll`. SSE endpoints for site events, progress, preview, and mount remain available and consume the same KV-backed live bus. Arming **ESTOP** also emits an agent wake so the Windows poller shortens its wait and retrieves the stop sequence promptly.

---



## 18. Cron and the Windows agent

Cron bearer routes force reconcile and run cleanup. The agent may call the reconcile URL with `POM-FRET_CRON_SECRET` as a backup. On the observatory PC, `nina_agent.py` polls the main sequence endpoint on the order of tens of seconds (longer when idle), polls **ESTOP** delivery every few seconds while NINA is running, posts an agent pulse while a sequence is active, and uploads live preview JPEGs when the frame changes. Unchanged sequence fingerprints are skipped if NINA is already running. New work writes a temporary JSON file, powers PDU outlets for mount and camera with a warmup interval, and launches NINA with exit-after-sequence. On exit, the agent stacks or zips as configured, uploads to R2, and reports files to the cloud.

---

## 19. Production configuration

Production imaging requires Upstash KV credentials, the imaging queue secret, the cron secret, and full R2 access (endpoint, bucket, and keys). Observatory-facing settings include the NINA progress Basic password, optional progress username, session end marker, mount telemetry secret, and--off Vercel--an optional observatory status file path. Site mail uses Resend. Bootstrap administrators are listed in `BOOTSTRAP_ADMIN_EMAILS`. Optional settings cover file-backed queue fallback, R2 write secret, presign TTL, object prefix, Astrometry, and radar upstream.

The Windows agent environment mirrors the queue secret, cron secret, R2 credentials, and PDU username and password.

---

## 20. HTTP API surface

Public authentication routes cover signup, login, logout, current user, password change, and email verification. Admin routes manage members, imaging access and large-project approvals, equipment, and gallery moderation.

The imaging API is larger. Members create and inspect queue rows; the agent updates them and consumes `nina-sequence`. Progress, pulse, preview, session-files, download, and R2 mapping connect NINA and storage to the cloud. Administrators control sessions, **ESTOP**, schedule closed windows, and read the audit log. Cron forces reconcile and cleanup. Observatory status is publicly readable and patchable by administrators. Weather helpers expose tonight's prediction column, 7Timer astro conditions, and the storm-approach ring used by safety ESTOP. Member gallery and saved-session routes support the account experience. Proxies forward GOES imagery, radar tiles, moon SVS frames, and plate-solving without exposing upstream keys to the browser.

Route paths follow the `/api/...` tree in the repository. Authorization for each family follows the trust boundaries in Section 4.

---

## 21. Development, testing, and deployment

Unit tests run with `npm test` (tsx over the `lib/**/*test.ts` tree) and cover weather gates, planner holds, ESTOP sync, planned-start due logic, moon avoidance, reconcile fingerprints, and related helpers. `npm run test:types` performs typechecking. Locally, `npm run dev` serves the site; KV is optional when a file-backed queue is configured. Production deploys with `npm run deploy` (vercel `--prod --yes`) to the project aliased as `www.pomfretastro.org`. Supporting scripts sync Stellarium sky data, rebuild the variable-star shortlist, and configure R2 CORS for gallery uploads.

Administrator activity is appended to a bounded audit log with kinds such as emergency stop, end night, observatory transitions, queue status changes, holds, progress, plan changes, and NINA delivery. Administrators read it through the audit-log API.

---

## 22. Delivery priority and weather gate summary

When the agent requests work, delivery follows this order. An undelivered **ESTOP** sequence is delivered immediately. While **ESTOP** blocks, or while NINA is freshly running, ordinary imaging is refused. Without **Ready**, before the planned start, below **30 deg** altitude, or inside an admin closed window, candidates are skipped or refused. If **end night** is due and no imaging remains, the end-night template is delivered. Otherwise a due, **Ready**, sufficiently high **scheduled** session is delivered as ordinary NINA JSON.

Relative to one another, the **Ready** gate allows ASC cloud under **20%**, while the schedule hourly cloud rule requires under **10%**. Schedule precip is stricter than **Ready** precip. **Weather-safety ESTOP** uses site precip above **20%**, the storm ring, and high-confidence ASC rain. Display-only 7Timer values never change those gates.

---